Refactoring

TL;DR
Refactoring bedeutet Änderung der inneren Struktur des Codes ohne Beeinflussung äußeren Verhaltens.
Mit Hilfe von Refactoring kann man Code Smells beheben, und Lesbarkeit, Verständlichkeit und Wartbarkeit von Software verbessern.
Es ist wichtig, immer nur einzelne Schritte zu machen und anschließend die Testsuite laufen zu lassen, damit nicht versehentlich Fehler oder Verhaltensänderungen beim Refactoring eingebaut werden.
Prinzipiell kann man Refactoring manuell mit Search&Replace durchführen, aber es bietet sich an, hier die IDE-Unterstützung zu nutzen. Es stehen verschiedene Methoden zur Verfügung, die nicht unbedingt einheitlich benannt sein müssen oder in jeder IDE vorkommen. Zu den häufig genutzten Methoden zählen *Rename*, *Extract*, *Move* und *Push Up/Pull Down*.

Videos

- VL Refactoring
- Demo Refactoring: Rename
- Demo Refactoring: Encapsulate
- Demo Refactoring: Extract Method
- Demo Refactoring: Move Method
- Demo Refactoring: Pull up

Unterlagen

- PDF-Version

Was ist Refactoring?

> Refactoring ist, wenn einem auffällt, daß der Funktionsname `foobar` ziemlich bescheuert ist, und man die Funktion in `sinus` umbenennt.
>
> Quelle: "356: Refactoring" by Andreas Bogk on Lutz Donnerhacke: "Fachbegriffe der Informatik"

> Refactoring (noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behaviour.
>
> (Fowler 2011, p. 53)

Refactoring: Änderungen an der **inneren Struktur** einer Software

- Beobachtbares (äußeres) Verhalten ändert sich dabei **nicht**
  - Keine neuen Features einführen
  - Keine Bugs fixen
  - Keine öffentliche Schnittstelle ändern (*Anmerkung*: Bis auf Umbenennungen oder Verschiebungen von Elementen innerhalb der Software)
- Ziel: Verbesserung von Verständlichkeit und Änderbarkeit

Anzeichen, dass Refactoring jetzt eine gute Idee wäre

- Code “stinkt” (zeigt/enthält *Code Smells*)

  Code Smells sind strukturelle Probleme, die im Laufe der Zeit zu Problemen führen können. Refactoring ändert die innere Struktur des Codes und kann entsprechend genutzt werden, um die Smells zu beheben.

- Schwer erklärbarer Code

  Könnten Sie Ihren Code ohne Vorbereitung in der Abgabe erklären? In einer Minute? In fünf Minuten? In zehn? Gar nicht?

  In den letzten beiden Fällen sollten Sie definitiv über eine Vereinfachung der Strukturen nachdenken.

- Verständnisprobleme, Erweiterungen

  Sie grübeln in der Abgabe, was Ihr Code machen sollte?

  Sie überlegen, was Ihr Code bedeutet, um herauszufinden, wo Sie die neue Funktionalität anbauen können?

  Sie suchen nach Codeteilen, finden diese aber nicht, da die sich in anderen (falschen?) Stellen/Klassen befinden?

  Nutzen Sie die (neuen) Erkenntnisse, um den Code leichter verständlich zu gestalten.

> “Three strikes and you refactor.”
>
> (Fowler 2011, p. 58): “The Rule of Three”

Wenn Sie sich zum dritten Mal über eine suboptimale Lösung ärgern, dann werden Sie sich vermutlich noch öfter darüber ärgern. Jetzt ist der Zeitpunkt für eine Verbesserung.

Schauen Sie sich die entsprechenden Kapitel in (Passig und Jander 2013) und (Fowler 2011) an, dort finden Sie noch viele weitere Anhaltspunkte, ob und wann Refactoring sinnvoll ist.

Bevor Sie loslegen . . .

1. **Unit Tests** schreiben
   - Normale und ungültige Eingaben
   - Rand- und Spezialfälle
2. **Coding Conventions** einhalten
   - Sourcecode formatieren (lassen)
3. Haben Sie die fragliche Codestelle auch wirklich verstanden?!

Vorgehen beim Refactoring

Überblick über die Methoden des Refactorings

Die Refactoring-Methoden sind nicht einheitlich definiert, es existiert ein großer und uneinheitlicher “Katalog” an möglichen Schritten. Teilweise benennt jede IDE die Schritte etwas anders, teilweise werden unterschiedliche Möglichkeiten angeboten.

Zu den am häufigsten genutzten Methoden zählen

- Rename Method/Class/Field
- Encapsulate Field
- Extract Method/Class
- Move Method
- Pull Up, Push Down (Field, Method)

Best Practice

Eine Best Practice (oder nennen Sie es einfach eine wichtige Erfahrung) ist, beim Refactoring langsam und gründlich vorzugehen. Sie ändern die Struktur der Software und können dabei leicht Fehler oder echte Probleme einbauen. Gehen Sie also langsam und sorgsam vor, machen Sie einen Schritt nach dem anderen und sichern Sie sich durch eine gute Testsuite ab, die Sie nach jedem Schritt erneut ausführen: Das Verhalten der Software soll sich ja nicht ändern, d.h. die Tests müssen nach jedem einzelnen Refactoring-Schritt immer grün sein (oder Sie haben einen Fehler gemacht).

- Kleine Schritte: immer nur **eine** Änderung zu einer Zeit
- Nach **jedem** Refactoring-Schritt **Testsuite** laufen lassen

  => Nächster Refactoring-Schritt erst, wenn alle Tests wieder “grün”
- Versionskontrolle nutzen: **Jeden** Schritt **einzeln** committen

Refactoring-Methode: Rename Method/Class/Field

Motivation

Name einer Methode/Klasse/Attributs erklärt nicht ihren Zweck.

Durchführung

Name selektieren, “`Refactor > Rename`”

Anschließend ggf. prüfen

Aufrufer? Superklassen?

Beispiel

Vorher

```
public String getTeN() {}
```

Nachher

```
public String getTelefonNummer() {}
```

Refactoring-Methode: Encapsulate Field

Motivation

Sichtbarkeit von Attributen reduzieren.

Durchführung

Attribut selektieren, “`Refactor > Encapsulate Field`”

Anschließend ggf. prüfen

Superklassen? Referenzen? (Neue) JUnit-Tests?

Beispiel

Vorher

```
int cps;

public void printDetails() {
    System.out.println("Credits: " + cps);
}
```

Nachher

```
private int cps;

int getCps() { return cps; }
void setCps(int cps) {  this.cps = cps;  }

public void printDetails() {
    System.out.println("credits: " + getCps());
}
```

Refactoring-Methode: Extract Method/Class

Motivation

- Codefragment stellt eigenständige Methode dar
- “Überschriften-Code”
- Code-Duplizierung
- Code ist zu “groß”
- Klasse oder Methode erfüllt unterschiedliche Aufgaben

Durchführung

Codefragment selektieren, “`Refactor > Extract Method`” bzw. “`Refactor > Extract Class`”

Anschließend ggf. prüfen

- Aufruf der neuen Methode? Nutzung der neuen Klasse?
- Neue JUnit-Tests nötig? Veränderung bestehender Tests nötig?
- Speziell bei Methoden:
  - Nutzung lokaler Variablen: Übergabe als Parameter!
  - Veränderung lokaler Variablen: Rückgabewert in neuer Methode und Zuweisung bei Aufruf; evtl. neue Typen nötig!

Beispiel

Vorher

```java
public void printInfos() {
    printHeader();
    // Details ausgeben
    System.out.println("name:    " + name);
    System.out.println("credits: " + cps);
}
```

Nachher

```java
public void printInfos() {
    printHeader();
    printDetails();
}
private void printDetails() {
    System.out.println("name:    " + name);
    System.out.println("credits: " + cps);
}
```

Refactoring-Methode: Move Method

Motivation

Methode nutzt (oder wird genutzt von) mehr Eigenschaften einer fremden Klasse als der eigenen Klasse.

Durchführung

Methode selektieren, “`Refactor > Move`” (ggf. “Keep original method as delegate to moved method” aktivieren)

Anschließend ggf. prüfen

- Aufruf der neuen Methode (Delegation)?
- Neue JUnit-Tests nötig? Veränderung bestehender Tests nötig?
- Nutzung lokaler Variablen: Übergabe als Parameter!
- Veränderung lokaler Variablen: Rückgabewert in neuer Methode und Zuweisung bei Aufruf; evtl. neue Typen nötig!

Beispiel

Vorher

```java
public class Kurs {
    int cps;
    String descr;
}

public class Studi extends Person {
    String name;
    int cps;
    Kurs kurs;

    public void printKursInfos() {
        System.out.println("Kurs:    " + kurs.descr);
        System.out.println("Credits: " + kurs.cps);
    }
}
```

Nachher

```java
public class Kurs {
    int cps;
    String descr;

    public void printKursInfos() {
        System.out.println("Kurs:    " + descr);
        System.out.println("Credits: " + cps);
    }
```

```
}

public class Studi extends Person {
    String name;
    int cps;
    Kurs kurs;

    public void printKursInfos() { kurs.printKursInfos(); }
}
```

Refactoring-Methode: Pull Up, Push Down (Field, Method)

Motivation

- Attribut/Methode nur für die Oberklasse relevant: **Pull Up**
- Subklassen haben identische Attribute/Methoden: **Pull Up**
- Attribut/Methode nur für eine Subklasse relevant: **Push Down**

Durchführung

Name selektieren, “`Refactor > Pull Up`” oder “`Refactor > Push Down`”

Anschließend ggf. prüfen

Referenzen/Aufrufer? JUnit-Tests?

Beispiel

Vorher

```
public class Person { }

public class Studi extends Person {
    String name;
    public void printDetails() { System.out.println("name:    " + name); }
}
```

Nachher

```
public class Person { protected String name; }

public class Studi extends Person {
    public void printDetails() { System.out.println("name:    " + name); }
}
```

Wrap-Up

Behebung von **Bad Smells** durch **Refactoring**

=> Änderung der inneren Struktur ohne Beeinflussung des äußeren Verhaltens

- Verbessert Lesbarkeit, Verständlichkeit, Wartbarkeit
- Immer nur kleine Schritte machen
- Nach jedem Schritt Testsuite laufen lassen
- Katalog von Maßnahmen, beispielsweise *Rename*, *Extract*, *Move*, *Push Up/Pull Down*, …
- Unterstützung durch IDEs wie Eclipse, Idea, …

Zum Nachlesen

- Fowler (2011)
- Inden (2013, Kap. 11)

Lernziele

- k2: Begriff, Notwendigkeit und Vorgehen des/beim Refactoring
- k2: Bedeutung kleiner Schritte beim Refactoring
- k2: Bedeutung einer sinnvollen Testsuite beim Refactoring
- k2: Refactoring: Nur innere Struktur ändern, nicht äußeres Verhalten!
- k3: Anwendung der wichtigsten Refactoring-Methoden: Rename, Extract, Move, Push Up/Pull Down

Challenges

Betrachten Sie das Theatrical Players Refactoring Kata. Dort finden Sie im Unterordner java/ einige Klassen mit unübersichtlichem und schlecht strukturierten Code.
Welche *Bad Smells* können Sie hier identifizieren?
Beheben Sie die Smells durch die *schrittweise Anwendung* von den aus der Vorlesung bekannten Refactoring-Methoden. Denken Sie auch daran, dass Refactoring immer durch eine entsprechende Testsuite abgesichert sein muss - ergänzen Sie ggf. die Testfälle.

Quellen

Fowler, M. 2011. *Refactoring*. Addison-Wesley.
Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. dpunkt.verlag.
Passig, K., und J. Jander. 2013. *Weniger schlecht programmieren*. O'Reilly.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- “356: Refactoring” by Andreas Bogk on Lutz Donnerhacke: “Fachbegriffe der Informatik”

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)